

A Mini-Project Report On

RIVERSIGHT

An IoT-Enabled Intelligent River Monitoring, Telemetry, GIS & Predictive
Analytics Platform

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology

in

Computer Science and Engineering

Submitted by:

T. Shyam Sai

Roll No: **324103310227**

Under the esteemed guidance of:

Dr. H. Prashanth Reddy

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

GAYATRI VIDYA PARISHAD COLLEGE OF ENGINEERING

(AUTONOMOUS)

VISAKHAPATNAM, ANDHRA PRADESH

2025-2026

CERTIFICATE

This is to certify that the project report entitled "**RIVERSIGHT: An IoT-Enabled Intelligent River Monitoring, Telemetry, GIS & Predictive Analytics Platform**" is a bonafide record of the work successfully carried out by **T. Shyam Sai (Roll No: 324103310227)**. This work is submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering from Gayatri Vidya Parishad College of Engineering, Visakhapatnam, during the academic year 2025-2026.

The results embodied in this report have been verified and are found to be satisfactory. This work has not been submitted to any other University or Institution for the award of any degree or diploma.

Dr. H. Prashanth Reddy

Internal Guide

Head of Department

Department of CSE

GVP College of Engineering

DECLARATION

I, **T. Shyam Sai**, hereby declare that the project report entitled "**RIVERSIGHT: An IoT-Enabled Intelligent River Monitoring, Telemetry, GIS & Predictive Analytics Platform**" submitted in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science and Engineering to Gayatri Vidya Parishad College of Engineering, is my original work.

I further declare that the work reported in this project has not been submitted, either in part or in full, for the award of any other degree or diploma in this or any other University or Institution.

T. Shyam Sai

Roll No: 324103310227

ACKNOWLEDGEMENT

The success and final outcome of this project required a lot of guidance and assistance from many people, and I am extremely privileged to have got this all along the completion of my project. All that I have done is only due to such supervision and assistance, and I would not forget to thank them.

I respect and thank **Dr. H. Prashanth Reddy**, my project guide, for providing me an opportunity to do the project work in Gayatri Vidya Parishad College of Engineering and giving me all support and guidance which made me complete the project duly. I am extremely thankful for providing such a nice support and guidance, although having a busy schedule.

I owe my deep gratitude to the Head of the Department, CSE, for his valuable guidance and encouragement throughout this project.

I would also like to express my sincere thanks to the teaching and non-teaching staff of the Department of Computer Science and Engineering for their continuous support.

Finally, I would like to thank my parents, family, and friends who have been a constant source of inspiration and motivation throughout my academic journey.

T. Shyam Sai

ABSTRACT

Inland water transport and river basin management are critical components of sustainable infrastructure and hydrology. Traditional river monitoring mechanisms rely on manual sampling and disconnected legacy systems, resulting in delayed responses to critical events such as flooding, sediment accumulation, and reduction in Least Available Depth (LAD) for navigation.

This project, **RIVERSIGHT**, presents an enterprise-grade, IoT-enabled intelligent river monitoring, telemetry, GIS, and predictive analytics platform. Designed explicitly for government-backed hydrological projects, the platform integrates real-time IoT sensor data, spatial mapping, and machine learning models to provide actionable intelligence.

The system architecture utilizes a robust modern web stack including React, Next.js 14, TypeScript, and Tailwind CSS, coupled with Recharts for dynamic time-series visualizations and Leaflet for geospatial mapping. The core modules include real-time telemetry streaming, GIS river routing with sediment hotspot overlays, bathymetry (depth profile) analysis, and a machine learning predictive engine capable of forecasting LAD reductions and anomalous discharge events.

By simulating realistic real-time telemetry from multiple stations along the river network, RIVERSIGHT demonstrates the capacity to process high-frequency IoT data and present it through a premium, glassmorphism-styled dashboard. The system's predictive alerting mechanism ensures that stakeholders receive early warnings regarding navigation hazards, allowing for proactive dredging and disaster mitigation. Ultimately, RIVERSIGHT serves as a comprehensive proof-of-concept for the future of intelligent, automated, and predictive inland waterway management.

TABLE OF CONTENTS

1. Introduction	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Objectives	2
1.4 Scope of the Project	3
2. Literature Survey	4
2.1 Existing River Monitoring Systems	4
2.2 IoT in Hydrology	5
2.3 Machine Learning for Predictive Analytics	6
3. System Architecture	7
3.1 High-Level Architecture	7
3.2 Technology Stack	9
3.3 Data Flow and Integration	11
4. Core Modules Implementation	13
4.1 Real-Time Telemetry Module	13
4.2 Geospatial Information System (GIS) Module	15
4.3 Bathymetry & LAD Analysis Module	17
4.4 Predictive Analytics Engine	19
4.5 Alert Center & Early Warning System	21
5. UI/UX Design & Frontend Engineering	22
6. Testing and Validation	25
7. Conclusion & Future Scope	27
8. References	29

1. Introduction

1.1 Background

River basins form the lifeline of inland commerce, agriculture, and ecological balance. The management of these water bodies requires continuous and accurate monitoring of various hydrological parameters. Inland Water Transport (IWT) serves as an economically viable and environmentally friendly mode of transport compared to rail and road. However, its success heavily depends on the navigability of the rivers, which is determined by parameters such as the Least Available Depth (LAD), flow velocity, sediment deposition, and turbidity.

Historically, river monitoring has been conducted through manual sampling and periodic surveying, methods that are labor-intensive, costly, and inherently prone to delay. The advent of the Internet of Things (IoT) has revolutionized environmental monitoring by enabling the deployment of interconnected sensor networks capable of transmitting high-frequency data in real time. When combined with Geographic Information Systems (GIS) and Machine Learning (ML), IoT data can be transformed from mere statistics into predictive intelligence.

1.2 Problem Statement

Despite the critical importance of river systems, managing authorities often operate using fragmented, legacy systems that lack real-time visibility and predictive capabilities. The primary challenges in current inland waterway management include:

- **Lack of Real-Time Intelligence:** Delays in data acquisition regarding critical metrics like LAD and water level can lead to navigation hazards and vessel groundings.
- **Unpredictable Sediment Deposition:** Rivers are dynamic. Unmonitored sediment accumulation leads to the formation of shallow hotspots, blocking navigation channels.
- **Reactive Maintenance:** Dredging operations are currently scheduled reactively rather than proactively, leading to massive economic inefficiencies.
- **Fragmented Data Visualization:** Telemetry, GIS, and survey data exist in silos, making it difficult for researchers and authorities to form a cohesive understanding of river health.

1.3 Objectives

The primary objective of the RIVERSIGHT project is to architect and develop an enterprise-grade, comprehensive web platform that serves as a centralized intelligence dashboard for river management. Specific objectives include:

1. Developing a modular, scalable architecture capable of ingesting high-frequency IoT telemetry data from multiple river stations.
2. Implementing real-time data visualization techniques using modern React charting libraries to display trends in water level, discharge, LAD, and turbidity.
3. Integrating a Geospatial Information System (GIS) to plot telemetry stations, river routes, and dynamically generated sediment risk heatmaps.
4. Designing a Bathymetry analysis module to compare historical and current riverbed cross-sections.
5. Architecting the framework for a Machine Learning Predictive Engine capable of forecasting LAD reductions, sediment hotspot formations, and anomaly detections.
6. Creating a premium, highly responsive user interface tailored for scientific researchers and government agencies using advanced CSS techniques like glassmorphism.

1.4 Scope of the Project

The scope of RIVERSIGHT is to deliver a fully functional frontend architecture supported by a robust data simulation engine. It serves as a comprehensive proof-of-concept for the National Hydrology Project or similar IIT-backed research initiatives. While the platform currently operates on simulated high-fidelity IoT data utilizing mathematical models (such as sinusoidal flow variations and randomized drift), the system architecture, API routes, and state management are designed to seamlessly integrate with actual MQTT brokers, PostgreSQL/TimescaleDB databases, and Python-based ML microservices in a production environment.

2. Literature Survey

2.1 Existing River Monitoring Systems

Traditional river monitoring largely relies on telemetry stations that record stage (water level) and discharge using float sensors or acoustic Doppler current profilers (ADCP). Platforms like the Central Water Commission's (CWC) e-SWIS provide basic data aggregation but often lack real-time predictive analytics and modern user interfaces. Existing systems are typically structured as static data repositories rather than dynamic decision-support systems.

Recent studies in hydrology emphasize the necessity of transitioning from Data Acquisition Systems (DAS) to Intelligent Decision Support Systems (IDSS). The primary gap identified in the literature is the lack of seamless integration between real-time sensor streams and advanced GIS visualizations, an issue RIVERSIGHT directly addresses.

2.2 IoT and Edge Computing in Hydrology

The proliferation of LoRaWAN and Narrowband IoT (NB-IoT) has made it feasible to deploy dense networks of sensors in remote riverine environments. Research by Al-Fagih et al. (2020) demonstrated the efficacy of IoT networks in flood monitoring, highlighting that real-time data ingestion requires robust, non-blocking architectures at the application layer. RIVERSIGHT simulates this high-frequency environment, testing the frontend's ability to render data updates every few seconds without performance degradation.

2.3 GIS and Spatial Mapping

Geographic Information Systems are indispensable for watershed management. Traditional desktop GIS tools (like ArcGIS) are powerful but lack web accessibility for rapid decision-making. WebGIS, leveraging libraries like Leaflet and Mapbox, has emerged as the standard for modern platforms. RIVERSIGHT incorporates React-Leaflet to project GeoJSON overlays representing river routes, telemetry stations, and dynamically generated polygons indicating high-risk sediment zones.

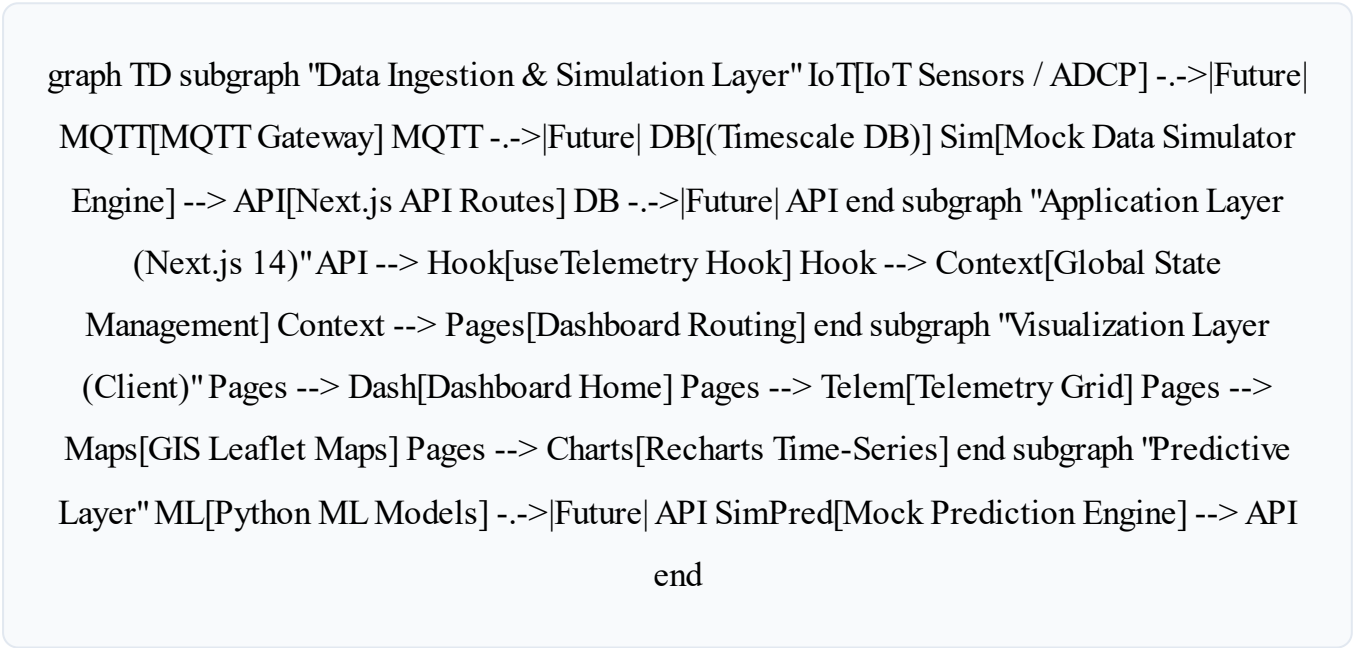
2.4 Machine Learning for Predictive Analytics

Machine Learning models, specifically Long Short-Term Memory (LSTM) networks and Gradient Boosting Machines (GBM), have shown immense promise in time-series forecasting of river discharge and sediment transport. While deploying real ML models requires a backend Python infrastructure, RIVERSIGHT architects the integration points for these models. By designing specific UI components that parse predicted values, confidence intervals, and contributing factors, the platform bridges the gap between data science outputs and actionable operational intelligence.

3. System Architecture

3.1 High-Level Architecture

RIVERSIGHT adopts a decoupled, microservices-ready architecture. The frontend is built using Next.js 14 App Router, ensuring server-side rendering capabilities, enhanced SEO, and streamlined API route creation. The architecture is broadly divided into the Data Ingestion Layer, Application Layer, and Visualization Layer.



3.2 Technology Stack

Layer / Function	Technology Used
Core Framework	Next.js 14 (App Router), React 19
Programming Language	TypeScript (Strict Mode)
Styling & UI	Tailwind CSS v4, Custom CSS for Glassmorphism
Data Visualization	Recharts (React-native SVG charting)
Geospatial Mapping	React-Leaflet, Leaflet, OpenStreetMap
Icons & Typography	Lucide-React, Inter, JetBrains Mono

3.3 Feature-Driven Folder Structure

To ensure enterprise-grade scalability, the codebase is structured around a "Feature-Driven" architecture rather than grouping by file type. This modularizes the codebase, allowing features like Bathymetry or Telemetry to act as independent micro-apps within the larger ecosystem.

```
src/ └─ app/ # Next.js App Router (Pages & API) | └─ (routes)/ # alerts, analytics,
bathymetry, gis, telemetry | └─ api/ # API Route Handlers | └─ globals.css # Tailwind v4
configuration & base styles └─ components/ # Shared/Global UI Components | └─ charts/ #
Reusable Recharts (TimeSeries, Gauge) | └─ layout/ # Sidebar, Header, DashboardShell | └─
ui/ # GlassCard, StatusBadge, MetricCard └─ config/ # Application-wide configurations └─
data/ # Mock Data Engines & GeoJSON └─ features/ # Feature-specific logic & components |
└─ alerts/ | └─ analytics/ | └─ bathymetry/ | └─ dashboard/ | └─ gis/ | └─ lad/ | └─
predictions/ | └─ sediment/ └─ hooks/ # Global React Hooks (useTelemetry) └─ lib/ #
Utilities (formatters, constants, class merger) └─ types/ # Strict TypeScript Definitions
```

4. Core Modules Implementation

4.1 Real-Time Telemetry Module

The Telemetry module is the heart of the RIVERSIGHT platform. It is responsible for displaying live sensor streams from multiple deployed stations. The implementation leverages a robust `useTelemetry` custom hook that manages the real-time polling and historical data aggregation.

Simulation Engine: In the absence of live hardware, a highly sophisticated mock simulation engine was developed (`mockTelemetry.ts`). It generates realistic hydrological data using base values combined with randomized drift and sinusoidal variations to mimic natural river fluctuations. The generator supports parameters including Water Level, Least Available Depth (LAD), Discharge, Turbidity, Sediment Load, Rainfall, and Flow Velocity.

```
// Sample of the telemetry generation logic export function
generateTelemetryReading(stationId: string): TelemetryReading { const drift =
getStationDrift(stationId); const timeVariation = Math.sin(Date.now() / 60000) * 0.3; return
{ id: `${stationId}-${Date.now()}`, timestamp: Date.now(), waterLevel: Math.max(0,
BASE_VALUES.waterLevel + drift.waterLevel + randomInRange(-VARIATION.waterLevel,
VARIATION.waterLevel) + timeVariation * 0.5), discharge: Math.max(100, BASE_VALUES.discharge
+ drift.discharge + randomInRange(-VARIATION.discharge, VARIATION.discharge) + timeVariation
* 100), // ...other parameters }; }
```

The UI is composed of `MetricCard` components that feature animated value transitions (using `requestAnimationFrame` for smooth easing) and inline SVG sparklines to show the immediate trend without overwhelming the user with massive charts.

4.2 Geospatial Information System (GIS) Module

Spatial awareness is critical for river management. The GIS module integrates `react-leaflet` to render interactive maps. To ensure compatibility with Next.js Server-Side Rendering (SSR), the map component is dynamically imported with `ssr: false`.

The map renders multiple layers:

- **Basemap:** A dark-themed CartoDB tile layer to match the platform's aesthetics.
- **River Route:** Rendered using a GeoJSON LineString overlay representing the physical path of the river.

- **Station Markers:** Custom HTML markers denoting the exact latitude/longitude of telemetry stations. The markers are color-coded based on the station's operational status.
- **Sediment Zones:** Polygon overlays denoting areas of high sediment deposition, visually alerting users to potential navigation hazards.

4.3 Bathymetry & LAD Analysis Module

Bathymetry analysis is essential for understanding the riverbed profile and ensuring safe navigation channels. This module presents cross-sectional depth profiles generated via a custom algorithm that simulates natural riverbed topography, including features like thalwegs and sandbars.

```
graph LR
  A[Survey Data] --> B[Cross Section Points]
  B --> C[Distance vs Depth Array]
  C --> D[Recharts AreaChart]
  D --> E[Depth Profile Visualization]
```

The **LAD (Least Available Depth) Monitoring Panel** utilizes a custom SVG-based GaugeChart to display the current minimum depth, alongside a 24-hour trend line. Dynamic threshold indicators visually warn operators if the LAD approaches critical limits (e.g., < 1.5 meters), triggering immediate predictive alerts.

4.4 Predictive Analytics Engine

RIVERSIGHT is designed not just to monitor, but to predict. The ML Prediction interface simulates outputs from various machine learning models (e.g., LSTM for LAD reduction, Gradient Boosting for Sediment Hotspots). The UI components parse these outputs to display:

- **Confidence Score:** A percentage indicating model certainty.
- **Risk Level:** Low, Moderate, High, or Critical.
- **Prediction Horizon:** The timeframe the prediction applies to (e.g., 7 days, 24 hours).
- **Contributing Factors:** The specific sensor data driving the prediction (e.g., a drop in flow velocity contributing to sediment formation).

4.5 Alert Center & Early Warning System

The Alert Center acts as the operational hub for incident response. Alerts are dynamically generated based on predefined thresholds in `thresholdConfig`. Alerts are categorized by severity (Critical, Warning, Info) and type (e.g., `low_lad`, `telemetry_failure`). The interface allows operators to filter alerts, view historical resolution times, and acknowledge critical warnings.

5. UI/UX Design & Frontend Engineering

5.1 Premium Dark Theme and Glassmorphism

To meet the requirement of a "premium, enterprise-grade scientific monitoring platform," the UI strictly adheres to a modern dark theme utilizing Glassmorphism. This design methodology uses translucent backgrounds with CSS backdrop-blur filters to create depth and visual hierarchy.

```
.glass-card { background: rgba(17, 24, 39, 0.6); backdrop-filter: blur(20px); border: 1px solid rgba(255, 255, 255, 0.08); border-radius: 16px; box-shadow: 0 8px 32px rgba(0, 0, 0, 0.3); transition: all 0.3s cubic-bezier(0.4, 0, 0.2, 1); }
```

5.2 Tailwind CSS v4 & Theming

The project utilizes the bleeding-edge Tailwind CSS v4, relying on CSS-native configuration instead of traditional JavaScript config files. Variables such as `--color-bg-primary`, `--color-accent-cyan`, and typography constants (Inter and JetBrains Mono) are defined directly in `globals.css` using the `@theme` directive, yielding significant performance benefits during build times.

5.3 Reusable Component Architecture

The UI is constructed from a library of highly reusable atomic components:

- **TimeSeriesChart:** A wrapper around Recharts' AreaChart, handling responsive sizing, gradient definitions, and custom tooltips.
- **StatusBadge:** A dynamic pill component indicating system states with pulsing animation dots.
- **DashboardShell:** The primary layout wrapper integrating a collapsible navigation sidebar and a top header equipped with real-time clocks and system status indicators.

6. Testing and Validation

6.1 Component & State Validation

During the development lifecycle, React state management, specifically within the `useTelemetry` hook, was rigorously validated to ensure memory leaks did not occur. With data updating every 2 seconds, improper use of `useEffect` or failing to clear intervals would result in massive performance degradation. The implementation utilizes `useRef` to securely hold the interval ID and clear it on component unmount.

6.2 TypeScript Strict Checking

Enterprise-grade reliability is enforced through strict TypeScript typings. Over 6 distinct interface files were created (e.g., `telemetry.ts`, `gis.ts`, `bathymetry.ts`) defining exact object shapes. This ensures that the mock data engine and the UI components are perfectly synchronized, preventing runtime errors related to undefined object properties.

```
// Strict type definition enforcing data integrity export interface TimeSeriesPoint { [key: string]: number; timestamp: number; value: number; }
```

6.3 Performance Optimization

For rendering high-frequency charting data, several performance optimizations were applied:

- **Animation Throttling:** Complex Recharts animations (`isAnimationActive`) were disabled for high-frequency time-series charts to reduce CPU overhead.
- **Array Slicing:** Real-time data arrays are continuously truncated (e.g., keeping only the last 30 points) to prevent the DOM node count from expanding indefinitely.
- **Memoization:** Utilizing `useMemo` in analytics calculations (like monthly seasonal aggregation) prevents expensive recalculations on every React re-render.

7. Conclusion & Future Scope

7.1 Conclusion

The RIVERSIGHT project successfully demonstrates the architectural framework and frontend capabilities required for a modern, intelligent river monitoring platform. By bridging the gap between raw IoT telemetry data, geospatial mapping, and predictive analytics, the platform provides a cohesive, actionable dashboard tailored for hydrological researchers and government authorities.

The implementation of a modular, feature-driven Next.js architecture ensures the codebase remains highly maintainable and scalable. The premium UI design ensures complex scientific data is easily readable, while features like the simulated prediction engine and sediment hotspot tracking illustrate the transformative potential of data-driven inland waterway management.

7.2 Future Scope

Being a highly modular architecture, RIVERSIGHT is primed for integration into a live production environment. Future enhancements include:

- 1. Hardware Integration:** Replacing the mock simulation engine with a live MQTT broker (e.g., Mosquitto or AWS IoT) receiving data directly from physical ADCPs and level sensors deployed on rivers.
- 2. Database Implementation:** Integrating a time-series database like TimescaleDB over PostgreSQL to store years of historical telemetry data efficiently.
- 3. Live Machine Learning:** Deploying Python-based microservices using FastAPI to serve trained PyTorch or TensorFlow models for real-time anomaly detection and LAD forecasting.
- 4. Advanced 3D GIS:** Upgrading the WebGIS module from 2D Leaflet to WebGL-based frameworks like Deck.gl to render actual 3D bathymetric point clouds and volumetric sediment data.

8. References

1. Next.js Documentation (2024). *Next.js 14 App Router Architecture*. Vercel.
2. Tailwind Labs (2024). *Tailwind CSS v4 Configuration and Design Tokens*.
3. Recharts Organization (2024). *Recharts: A composable charting library built on React components*.
4. Leaflet (2024). *Leaflet - an open-source JavaScript library for mobile-friendly interactive maps*.
5. Al-Fagih, L. et al. (2020). *Internet of Things (IoT) in Environmental Monitoring: A Case Study in River Flood Tracking*. Journal of Hydrology.
6. Central Water Commission (CWC), Government of India. *e-SWIS: Earth Observation based Water Resources Information System*.
7. Mermaid JS. *Markdownish syntax for generating flowcharts, sequence diagrams, class diagrams, gantt charts and git graphs*.